Homework 1 – Version B: Orders at Bingo Supermarket

Course: Mobile Application and Service Development **Level:** standard · **Points:** 7 · **Deadline:** 30.10.2026 at 23:59 · **Defense:** lab session, 02.11.2026

Description

You are a purchasing agent at Bingo supermarket and you order goods from suppliers. The orders are written as text. You write a Kotlin program that reads them, turns them into objects and prints an order report. You design the data model yourself. At the defense you will justify your model.

Input data

Each order is on one line. Fields are separated by `;`: order code, supplier, product, category, quantity (pieces), unit price in KM and payment method. For deferred payment, the number of days of deferral follows the colon.

Categories in the data: `FOOD`, `BEVERAGES`, `HYGIENE`. Payment methods: `ADVANCE` (advance payment), `TRANSFER` (bank transfer), `DEFERRED` (deferred payment).

Copy this text into your code as a String:

```
N01; Mepas; Kandit milk chocolate 100 g; FOOD; 200; 1.50; TRANSFER
N02; Orbico; Head & Shoulders shampoo 400 ml; HYGIENE; 60; 7.90; ADVANCE
N03; MCI; Jaffa biscuits 150 g; FOOD; 300; 1.20; TRANSFER
N04; Orbico; Ariel detergent 3 kg; HYGIENE; 40; 14.50; DEFERRED:60
N05; Mepas; Maraska sour cherry nectar 1 l; BEVERAGES; 120; 2.30; TRANSFER
N06; MCI; Nescafé Classic 200 g; BEVERAGES; 50; 9.80; DEFERRED:30
N07; Mepas; Kandit hazelnut chocolate 100 g; FOOD; 150; 1.80; ADVANCE
N08; Orbico; Pantene shampoo 400 ml; HYGIENE; 60; 6.50; TRANSFER
N09; MCI; Nivea cream 150 ml; HYGIENE; 80; 5.20; DEFERRED:45
N10; Mepas; Maraska apple juice 1 l; BEVERAGES; 100; 1.90; DEFERRED:15
N11; Orbico; Ariel detergent 3 kg; HYGIENE; forty; 14.50; TRANSFER
```

Model requirements

Your model must satisfy the following:

1. Two orders with the same data must be equal when compared with `==`, and printing an order with `println` must be readable, without writing that logic by hand.
2. Order data cannot be changed after the order is created.
3. If you misspell the product category in your code, the program must not even compile.
4. The order amount is not stored; it is always calculated from the quantity and the unit price.

5. There are three payment methods: advance payment, bank transfer and deferred payment. Only deferred payment has a number of days. Advance payment and bank transfer must not have a field for the number of days, neither empty nor set to zero.
6. If a new payment method is added later, the compiler itself must show every place in the code that needs to handle it.

Reading the text

- Write a function that creates an order from **one line** of text. If the line is not valid (wrong number of fields, unknown category, quantity or price that is not a number...), the function does not return an order and **the program keeps running**.
- Spaces around `;` must not cause problems.
- One line in this data is invalid. The program must skip it and continue. At the defense you will get other invalid lines too.

What the program must calculate

1. How many orders were loaded and how many lines are invalid.
2. The total amount of all orders.
3. The amount per category, in the order food, beverages, hygiene.
4. The number of orders per payment method, in the order advance payment, bank transfer, deferred payment.
5. The codes of orders with deferred payment longer than 30 days.
6. The order with the largest amount. If there are several, take the first one in the list.
7. Write a function that returns all orders for a given supplier. In `main`, call it for the supplier **MCI** and print each order on its own line, in the format shown in the expected output.

Code rules

- Queries over the list are done with **collection functions**, not `for` / `while` loops. You may use a loop or `forEach` only for printing.
- Calculate the amount per category by **grouping** the orders, not with a separate query for each category.
- Printing one order is done by an **extension function** that you write yourself. For deferred payment, the output also shows the number of days.
- The `!!` operator is not allowed.

Expected output

Your program must print this (print amounts with two decimals):

```
=== BINGO: SUPPLIER ORDERS ===
Orders loaded: 10
Invalid lines: 1
Total amount: 3746.00 KM

Amount per category:
  Food: 930.00 KM
  Beverages: 956.00 KM
  Hygiene: 1860.00 KM

Orders per payment method:
  Advance payment: 2
  Bank transfer: 4
  Deferred payment: 4

Deferral longer than 30 days: N04, N09
Largest order: N04 Ariel detergent 3 kg - 40 x 14.50 = 580.00 KM (deferred 60
days)

Orders for supplier: MCI
  N03 Jaffa biscuits 150 g - 300 x 1.20 = 360.00 KM (bank transfer)
  N06 Nescafé Classic 200 g - 50 x 9.80 = 490.00 KM (deferred 30 days)
  N09 Nivea cream 150 ml - 80 x 5.20 = 416.00 KM (deferred 45 days)
```

Work and submission rules

- **No AI.** Do not use ChatGPT or similar tools, and turn off AI completion in your editor. You write the code in IntelliJ IDEA; how to set it up and turn off AI is described in `INSTRUCTIONS.md`.
- **Submission via Classmoji**, in the repository you receive (steps in `INSTRUCTIONS.md`). You need **at least 5 commits** that follow your progress, e.g. model, reading the text, queries, output. One big commit at the end leads to fewer points.
- **Live defense in the lab, without AI.** Justify your model, explain your code, then do a small additional task in your own code.

Grading

- The homework is passed if all model requirements and code rules are met, the output is correct and the defense is successful.
- **Fewer points for:** incorrect output, a model that does not meet the requirements, breaking the code rules, a weak defense.
- **If at the defense the student cannot show that they wrote the code themselves, the homework is worth 0 points.**